

Support Vector Machine

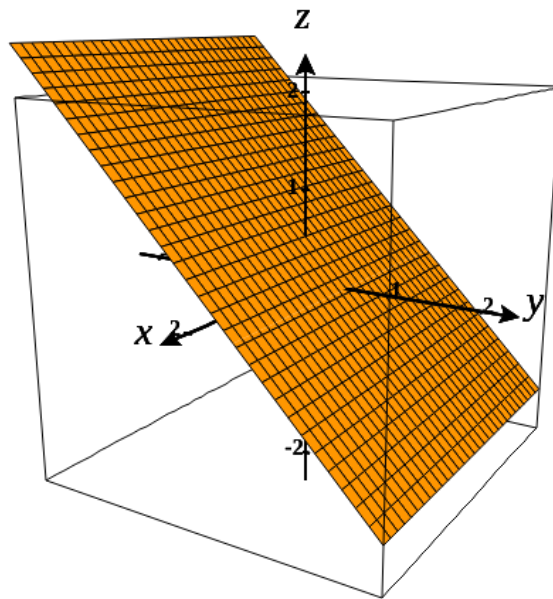
Es un modelo enfocado en la clasificación binaria. La idea detrás de este modelo es encontrar un hiperplano tal que separe completamente un conjunto de datos en dos grupos (en ML se conoce como *labels, classes or categories*)

Introducción

A continuación se describe que es un hiperplano y como se relaciona con el concepto de separabilidad lineal.

Definición de un hiperplano

En geometría, un **hiperplano** es un subespacio de una dimensión menos que su espacio ambiente (Kowalczyk, 2017 p. 24), es decir, si el espacio en el que nos encontramos tiene N dimensiones, entonces el hiperplano tendrá $N - 1$ dimensiones.



El subespacio geométrico cambia en cada espacio dimensional:

- en un espacio de 2 dimensiones, el hiperplano es una *línea*.
- en un espacio de 3 dimensiones, el hiperplano es un *plano*

- en un espacio mayor a tres dimensiones se denomina hiperplano como tal.

Matemáticamente se define por la siguiente *ecuación lineal*:

$$w_1x_1 + \dots + w_px_p + b = 0$$

en forma matricial, se expresa como:

$$\mathbf{w} \cdot \mathbf{x} + b = 0$$

- $\mathbf{x}$ el vector de variables independientes.

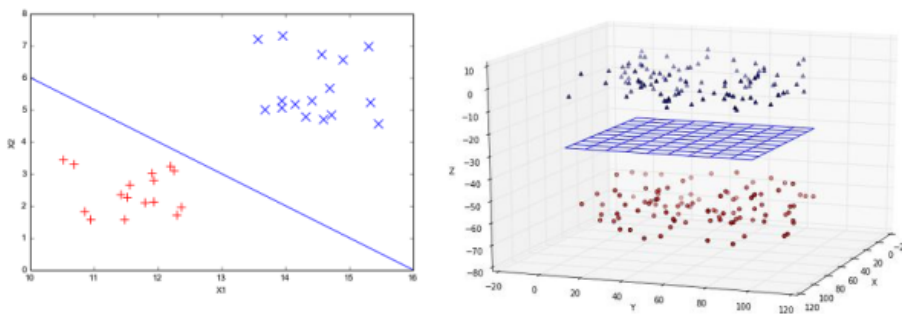
┌ Todos los puntos $\mathbf{x}_i$ sobre el hiperplano satisfacen la ecuación.

- $\mathbf{w}$ el vector de coeficiente asociado a las variables independientes (comúnmente llamado vector de pesos).
- b un escalar denominado término independiente (o el bias).

└ En resumen, un hiperplano está definido por un vector de pesos $\mathbf{w}$ y un término independiente b .

Separabilidad lineal de datos

Dado una conjunto de puntos $\mathcal{X} = \{\mathbf{x}_1, \dots, \mathbf{x}_n | \mathbf{x}_i \in \mathbb{R}^p\}$, se dice que $\mathcal{X}$ es linealmente separable si existe un *hiperplano* que separa los puntos en dos grupos.



Clasificación de datos con un hiperplano

El concepto básico de un modelo SVM es la clasificación de datos mediante un hiperplano, denominado **hiperplano separador**.

Sea $\mathcal{D}$ un conjunto de datos *linealmente separable*:

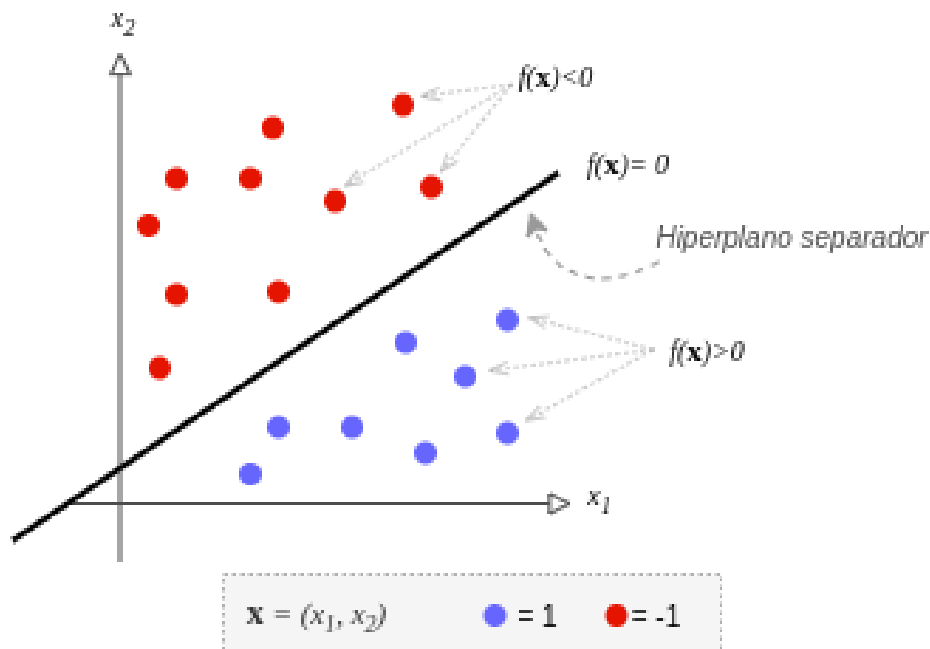
$$\mathcal{D} = \{(\mathbf{x}_i, y_i) | \mathbf{x}_i \in \mathbb{R}^p, y_i \in \{-1, 1\}\}_{i=1}^n$$

- $\mathbf{x}_i$ es un vector correspondiente a la i -ésima observación.
- y_i es la respuesta esperada asociada a $\mathbf{x}_i$.
- n es el número de muestras u observaciones.

También, supongamos un **hiperplano óptimo**¹ tal que separa completamente el conjunto de datos $\mathcal{D}$ en dos grupos (o clases), definido por la siguiente ecuación lineal:

$$\mathbf{w} \cdot \mathbf{x} + b = 0$$

Por comodidad, vamos a expresar el hiperplano como una función: $f(\mathbf{x}) = 0$



Dado este escenario, se puede observar que:

- Cualquier punto sobre el hiperplano cumple con: $f(\mathbf{x}_i) = 0$.
- Los puntos que están a los lados del hiperplano cumplen: $f(\mathbf{x}_i) > 0$ o $f(\mathbf{x}_i) < 0$.

Regla de clasificación

Evaluando cada observación $\mathbf{x}_i$ en la ecuación del hiperplano y utilizando una función sign , podemos estimar las respuestas $\hat{y}_i$:

$$\hat{y}_i = \text{sign}(f(\mathbf{x}_i)) = \begin{cases} +1, & \text{si } f(\mathbf{x}_i) > 0 \\ -1, & \text{si } f(\mathbf{x}_i) < 0 \end{cases}$$

como se puede observar solo necesitamos conocer el signo para hacer la clasificación.

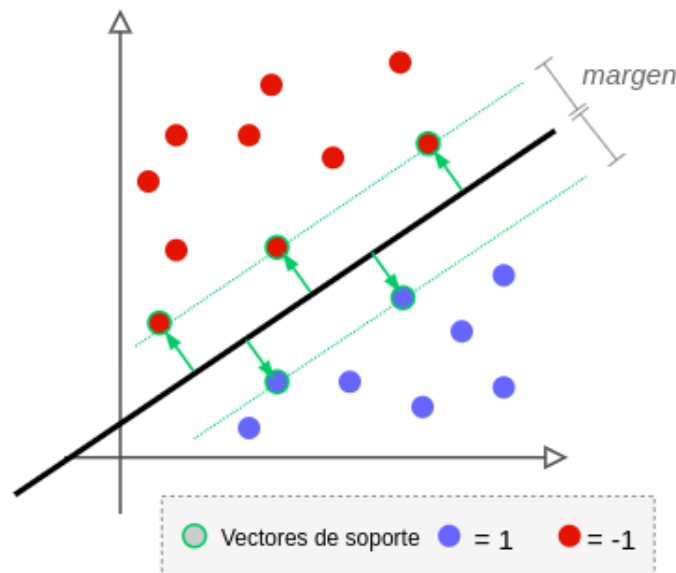
En el libro de Kowalczy (2017, p. 26) lo define como una función hipótesis:
$$h(\mathbf{x}_i) = \text{sign}(\mathbf{w} \cdot \mathbf{x}_i + b).$$

¿Cómo encontrar el mejor hiperplano separador?

El desafío es encontrar el **hiperplano óptimo** que separe los datos, lo que significa encontrar los valores de $\mathbf{w}$ y b que produzca un hiperplano que mejor separe los datos. Para esto se necesita resolver un *problema de optimización* ²

Problema de optimización de SVM

Para encontrar el mejor hiperplano separador debemos resolver un *problema de optimización* ² tal que la distancia (denominado *margen*) entre el hiperplano y los puntos más cercanos a este, sea lo más grande posible, en términos de optimización debemos **maximizar el margen**.



- Los puntos que están sobre el límite del margen (línea punteada) se denominan **vectores de soporte** (*support vectors*).

Recordemos que un hiperplano está definido por $\mathbf{w} \cdot \mathbf{x} + b = 0$, por lo tanto, encontrar el hiperplano óptimo es hallar los valores de $\mathbf{w}$ y b .

Derivación del hiperplano óptimo

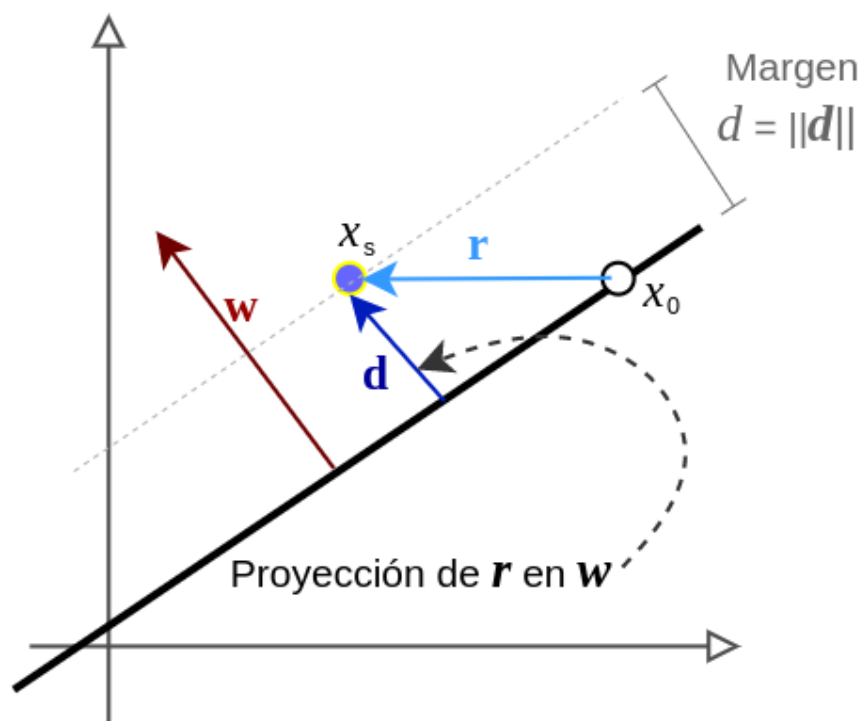
Como se mencionó anteriormente, debemos **maximizar el margen** entre el hiperplano y los puntos más cercanos a este. Para ello debemos hallar una **función objetivo**.

Planteamiento del Problema primal

Recordemos que $\mathcal{D} = \{(\mathbf{x}_i, y_i) | \mathbf{x}_i \in \mathbb{R}^p, y_i \in \{-1, 1\}\}_{i=1}^n$ es el conjunto de datos.

Sea $\mathbf{x}_0$ un punto sobre el hiperplano separador y $\mathbf{x}_s$ uno de los puntos del conjunto de datos $\mathcal{D}$ que sea el **más cercano al hiperplano**.

Tomando estos dos puntos, podemos formar un vector $\mathbf{r} = \mathbf{x}_s - \mathbf{x}_0$. Luego, por definición sabemos que el vector de pesos $\mathbf{w}$ es ortogonal al hiperplano, proyectando el vector $\mathbf{r}$ sobre el vector $\mathbf{w}$ obtenemos un vector $\mathbf{d}$ cuyo módulo (longitud) es la mínima distancia euclidiana entre $\mathbf{x}_s$ y el hiperplano, a esta distancia la denominamos *margen*



el cual se puede calcular mediante la siguiente fórmula:

$$d = \frac{\mathbf{w}}{\|\mathbf{w}\|} \mathbf{r}$$

sustituyendo $\mathbf{r}$ y haciendo un poco de manipulación algebraica, se puede llegar a la siguiente expresión:

$$d = \frac{|\mathbf{w} \cdot \mathbf{x}_s + b|}{\|\mathbf{w}\|}$$

- El valor absoluto evita valores negativos, lo cual puede ocurrir cuando los puntos están del otro lado.

recordemos que $\mathbf{x}_s$ es una observación del conjunto $\mathcal{D}$, entonces se cumple: $\mathbf{w} \cdot \mathbf{x}_s + b = \pm 1$, reemplazando esto en la anterior expresión, obtenemos:

$$d = \frac{1}{\|\mathbf{w}\|}$$

El procedimiento más extenso se puede ver en este [artículo](#).

Definida esta métrica, podemos plantear un **problema de optimización** que maximice el *margen*, esto es:

$$\begin{aligned} \max_{\mathbf{w}, b} \quad & \frac{1}{\|\mathbf{w}\|} \\ \text{sujeto a} \quad & y_i(\mathbf{w} \cdot \mathbf{x}_i + b) \geq 1, \quad i = 1, \dots, n \end{aligned}$$

Recordemos que $y_i \in \{-1, 1\}$ y suponiendo que el hiperplano separa correctamente a los puntos $\mathbf{x}_i$, entonces se cumple: $y_i(\mathbf{w} \cdot \mathbf{x}_i + b) \geq 1$.

Este problema de optimización se puede reescribir en términos de **mínimización**, para esto vamos a subir el término $\mathbf{w}$ al numerador. Además, **por conveniencia** vamos a elevar al cuadrado y dividir entre dos:

$$\begin{aligned} \min_{\mathbf{w}, b} \quad & \frac{1}{2} \|\mathbf{w}\|^2 \\ \text{sujeto a} \quad & y_i(\mathbf{w} \cdot \mathbf{x}_i + b) \geq 1, \quad i = 1, \dots, n \end{aligned}$$

En la literatura, la restricción se suele reescribir como: $y_i(\mathbf{w} \cdot \mathbf{x}_i + b) - 1 \geq 0, \quad i = 1, \dots, n$.

Ahora tenemos una expresión acorde a un **problema de optimización cuadrática convexa** el cual es mucho mas simple de resolver (Kowalczyk, 2017, p. 53).

El problema primal

Para resolver el problema de optimización de SVM se puede recurrir al método de **multiplicadores de Lagrange**, obteniendo la siguiente expresión (*Lagrangian function*):

$$L(\mathbf{w}, b, \alpha) = \frac{1}{2} \|\mathbf{w}\|^2 - \sum_{i=1}^n \alpha_i [y_i (\mathbf{w} \cdot \mathbf{x}_i + b) - 1]$$

- α_i es un multiplicador de *Lagrange* asociado a cada par $(\mathbf{x}_i, y_i)$.

ahora, el problema de optimización a resolver es:

$$\begin{aligned} \min_{\mathbf{w}, b} \quad & \max_{\alpha} \quad L(\mathbf{w}, b, \alpha) \\ \text{sueto a} \quad & \alpha_i \geq 0, \quad i = 1, \dots, n \end{aligned}$$

Esto se conoce como la definición del **problema primal** o el **problema del Lagrangiano**.

Notemos que L depende de tres variables $(\mathbf{w}, b, \alpha)$. Por lo tanto, el siguiente paso será encontrar una expresión equivalente que solamente dependa de los multiplicadores de *Lagrange*.

El Problema dual

A partir del problema primal, donde obtuvimos L (una función que depende de tres variables), ahora el objetivo será obtener una función que solo dependa de los α_i (multiplicadores de *Lagrange*).

Derivamos L respecto a $\mathbf{w}$ y b e igualamos a cero:

$$\begin{aligned} \frac{\partial L}{\partial \mathbf{w}} &= \mathbf{w} - \sum_{i=1}^n \alpha_i y_i \mathbf{x}_i = 0 \\ \frac{\partial L}{\partial b} &= - \sum_{i=1}^n \alpha_i y_i = 0 \end{aligned}$$

de las anteriores expresiones, obtenemos:

$$\mathbf{w} = \sum_{i=1}^n \alpha_i y_i \mathbf{x}_i \quad (1)$$

$$\sum_{i=1}^n \alpha_i y_i = 0 \quad (2)$$

reemplazando (1) y (2) en L y reordenando, obtenemos:

$$\mathcal{W}(\alpha) = \sum_{i=1}^n \alpha_i - \frac{1}{2} \sum_{i=1}^n \sum_{j=1}^n \alpha_i \alpha_j y_i y_j \mathbf{x}_i \cdot \mathbf{x}_j$$

finalmente, el problema de optimización a resolver es:

$$\begin{aligned} \max_{\alpha} \quad & \sum_{i=1}^n \alpha_i - \frac{1}{2} \sum_{i=1}^n \sum_{j=1}^n \alpha_i \alpha_j y_i y_j \mathbf{x}_i \cdot \mathbf{x}_j \\ \text{sujeto a} \quad & \alpha_i \geq 0, \text{ for any } i = 1, \dots, n \\ & \sum_{i=1}^n \alpha_i y_i = 0 \end{aligned}$$

- se añade la restricción $\sum_{i=1}^n \alpha_i y_i = 0$ para eliminar b de la función.

De esta forma obtenemos una función que solo depende de los multiplicadores de Lagrange.

*Esto se conoce como la definición del **problema dual**, también conocido como **problema dual de Wolfe**.*

Notemos que la función $\mathcal{W}$ es cuadrática en α y está sujeta a restricciones de desigualdad e igualdad afines, por lo tanto se trata de problema de optimización del tipo **programación cuadrática** (QP).

Existen librerías como the Python package called CVXOPT o CVXPY para resolver problemas de programación cuadrática.

Vectores de soporte

En términos generales, los vectores de soporte son aquellos puntos que están más cerca al *límite de decisión* (hiperplano). Por ende, influyen directamente en el límite de decisión.

Recordemos que cada observación $(\mathbf{x}_i, y_i)$ tiene asociado un α_i .

Los vectores de soporte son aquellos puntos cuyo $\alpha_i > 0$ (Kowalczy, 2017, p. 59).

Geométricamente:

- Los puntos con $\alpha_i > 0$ están sobre el límite del margen.
- Los puntos con $\alpha_i = 0$ están alejados del margen y del lado correcto.

En la práctica, los vectores de soporte son aquellos que tienen un α mayor a un número casi cero:


```
# Support vectors have positive multipliers.  
has_positive_multiplier = multipliers > 1e-7  
sv_multipliers = multipliers[has_positive_multiplier]
```

Cálculo de $\mathbf{w}$ y b

Luego de resolver el problema dual, obtenemos un vector α (los multiplicadores de Lagrange). Sin embargo, recordemos que el objetivo es encontrar los valores de $\mathbf{w}$ y b (hiperplano separador).

Cálculo de $\mathbf{w}$

El valor de $\mathbf{w}$ se puede calcular a partir de $\mathbf{w} = \sum_{i=1}^n \alpha_i y_i \mathbf{x}_i$.

Sin embargo, notemos que los puntos que no son vectores de soporte tienen $\alpha = 0$, entonces solo es necesario utilizar los vectores soporte:

$$\mathbf{w} = \sum_{s=1}^S \alpha_s y_s \mathbf{x}_s$$

- $\mathbf{x}_s$ es un vector de soporte.
- S es el número de vectores de soporte

Cálculo de b

Para obtener el valor de b recurrimos a una de las restricciones $y_i(\mathbf{w} \cdot \mathbf{x}_i + b) \geq 1$. Dado que nos interesa los puntos mas cercados al hiperplano, es decir, los que están sobre el límite del margen, utilizaremos la siguiente expresión $y_i(\mathbf{w} \cdot \mathbf{x}_i + b) = 1$.

Despejamos b :

$$b = y_i - \mathbf{w} \cdot \mathbf{x}_i$$

Sin embargo, en lugar de tomar un solo punto $\mathbf{x}_i$, es mejor tomar el promedio de los vectores de soporte:

$$b = \frac{1}{S} \sum_{s=1}^S (y_s - \mathbf{w} \cdot \mathbf{x}_s)$$

- S es el número de vectores de soporte.

Función de predicción

Recordemos la regla de clasificación: $\hat{y}_i = \text{sign}(\mathbf{w} \cdot \mathbf{x}_i + b)$.

Reemplazando $\mathbf{w} = \sum_{s=1}^S \alpha_s y_s \mathbf{x}_s$ en la regla de clasificación, obtenemos una función de predicción que utiliza los vectores de soporte:

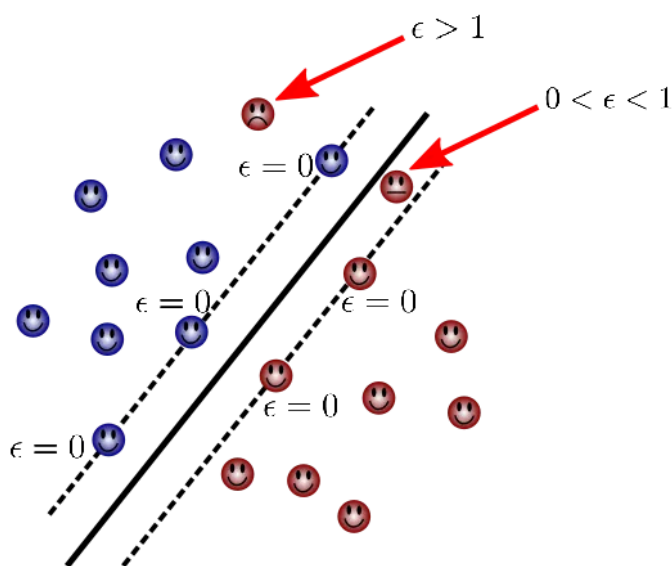
$$\hat{y}_j = \text{sign} \left(\sum_{s=1}^S \alpha_s y_s (\mathbf{x}_s \cdot \mathbf{x}_j) + b \right)$$

- $\mathbf{x}_j$ es el nuevo punto.
- $\mathbf{x}_s$ son los vectores de soporte.
- S es el número de vectores de soporte.

Esta formulación de SVM es llamado **hard margin SVM** porque solo trabaja con datos linealmente separables (Kowalczy, 2017, p. 64).

Soft margin SVM

La formulación *hard margin SVM* tiene una limitante práctica porque requiere que los datos sean linealmente separables, es decir, no permite errores, lo cual no ocurre en problemas de la vida real.



Para permitir errores debemos introducir una nueva variable de error ϵ , recordemos la restricción $y_i(\mathbf{w} \cdot \mathbf{x}_i + b) \geq 1$, ahora se convierte en:

$$y_i(\mathbf{w} \cdot \mathbf{x}_i + b) \geq 1 - \epsilon_i$$

Considerando esta nueva expresión dentro del **problema de optimización inicial**, tenemos:

$$\begin{aligned} \min_{\mathbf{x}, b, \epsilon} \quad & \frac{1}{2} \|\mathbf{w}\|^2 + C \sum_{i=1}^n \epsilon_i \\ \text{sujeto a} \quad & y_i(\mathbf{w} \cdot \mathbf{x}_i + b) \geq 1 - \epsilon_i \\ & \epsilon_i \geq 0 \quad i = 1, \dots, n \end{aligned}$$

- C es un *parámetro de regularización*, el cual define cuan estricto o permisivo será el margen.

Si C es muy grande casi no se permiten errores, lo conducirá aun limite ondulado sobreajustado al espacio de características original.

Si C es pequeño, se permite que las observaciones puedan estar del lado incorrecto, haciendo que limite sea más suave.

"The optimal value for C can be estimated by cross-validation"

-- Hastie, 2008, p. 421, 421, Kowalczy, 2017, p. 70

- la restricción $\epsilon_i \geq 0$ evita valores negativos para ϵ_i .

Aplicando *multiplicadores de Lagrange*, obtenemos:

$$L(\mathbf{w}, b, \alpha, \epsilon) = \frac{1}{2} \|\mathbf{w}\|^2 + C \sum_{i=1}^n \epsilon_i - \sum_{i=1}^n \alpha_i [y_i(\mathbf{w} \cdot \mathbf{x}_i + b) - 1 + \epsilon_i] - \sum_{i=1}^n \mu_i \epsilon_i$$

- μ_i es un nuevo multiplicador de Lagrange para los errores ϵ_i .

Luego, el **problema primal** se expresa mediante:

$$\begin{aligned} \min_{\mathbf{w}, b, \epsilon} \quad & \max_{\alpha} \quad L(\mathbf{w}, b, \alpha, \epsilon) \\ \text{sujeto a} \quad & \alpha_i \geq 0, \quad \mu_i \geq 0 \quad i = 1, \dots, n \end{aligned}$$

Siguiendo el mismo procedimiento del *problema dual*. Calculamos las derivadas de L respecto $\mathbf{w}$, b y ϵ e igualamos a cero:

$$\begin{aligned}\frac{\partial L}{\partial \mathbf{w}} &= \mathbf{w} - \sum_{i=1}^n \alpha_i y_i \mathbf{x}_i = 0 \\ \frac{\partial L}{\partial b} &= - \sum_{i=1}^n \alpha_i y_i = 0 \\ \frac{\partial L}{\partial \epsilon_i} &= C - \alpha_i - \mu_i = 0\end{aligned}$$

del cálculo anterior obtenemos tres expresiones:

$$\mathbf{w} = \sum_{i=1}^n \alpha_i y_i \mathbf{x}_i, \quad \sum_{i=1}^n \alpha_i y_i = 0, \quad \alpha_i = C - \mu_i$$

reemplazando estas expresiones en la función L y reordenandolo, obtenemos la siguiente función que solo depende de α :

$$\mathcal{W}(\alpha) = \sum_{i=1}^n \alpha_i - \frac{1}{2} \sum_{i=1}^n \sum_{j=1}^n \alpha_i \alpha_j y_i y_j \mathbf{x}_i \cdot \mathbf{x}_j$$

Finalmente, obtenemos el **problema dual** a optimizar:

$$\begin{aligned}\max_{\alpha} \quad & \sum_{i=1}^n \alpha_i - \frac{1}{2} \sum_{i=1}^n \sum_{j=1}^n \alpha_i \alpha_j y_i y_j \mathbf{x}_i \cdot \mathbf{x}_j \\ \text{sujeto a} \quad & 0 \leq \alpha_i \leq C, \quad \text{for any } i = 1, \dots, n \\ & \sum_{i=1}^n \alpha_i y_i = 0\end{aligned}$$

Recordemos las restricciones del problema primal $\alpha_i \geq 0$ y $\mu_i \geq 0$,

Reemplazando la expresión $\alpha_i = C - \mu_i$ y haciendo un poco de álgebra sobre las restricciones del problema primal, obtenemos: $0 \leq \alpha_i \leq C$.

Notemos que esta expresión es la misma que se obtuvo en el planteamiento del problema dual, excepto que ahora α_i tiene un límite superior en C .

Vectores de soporte

Al igual que en hard margin, los vectores de soporte son aquellos puntos $\mathbf{x}_i$ con $\alpha_i > 0$.

Geométricamente:

- *Los puntos con $\alpha = 0$ están del lado incorrecto*
- *Los puntos con $0 < \alpha_i < C$ están sobre el límite del margen.*

- Los puntos con $\alpha_i = 0$ están alejados del margen y del lado correcto.

Función Kernel

Notemos que la función $\mathcal{W}(\alpha)$ tiene un producto escalar de dos vectores $\mathbf{x}_i \cdot \mathbf{x}_j$, este puede ser reescrito como una **función kernel**:

$$K(\mathbf{x}_i, \mathbf{x}_j) = \mathbf{x}_i \cdot \mathbf{x}_j$$

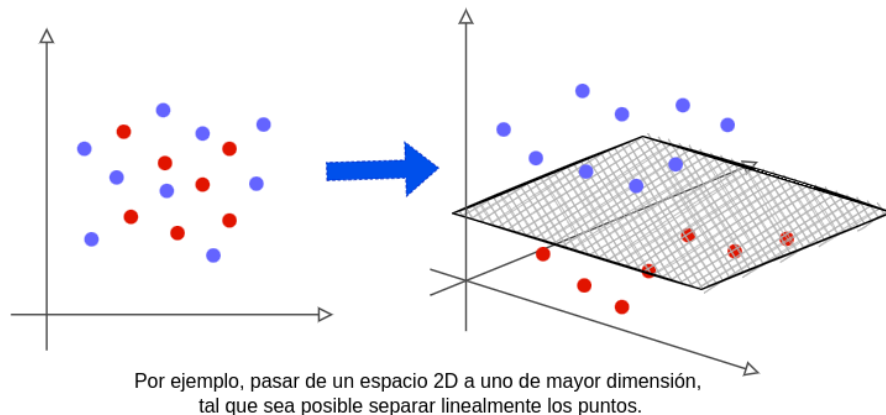
El truco del kernel

A partir de la definición del kernel K , podemos reescribir el **problema dual** del *soft margin* como:

$$\begin{aligned} \max_{\alpha} \quad & \sum_{i=1}^n \alpha_i - \frac{1}{2} \sum_{i=1}^n \sum_{j=1}^n \alpha_i \alpha_j y_i y_j K(\mathbf{x}_i, \mathbf{x}_j) \\ \text{sujeto a} \quad & \alpha_i \leq 0 \leq C, \quad \text{for any } i = 1, \dots, n \\ & \sum_{i=1}^n \alpha_i y_i = 0 \end{aligned}$$

a este cambio se conoce como el "**truco del kernel**".

Este "truco" del kernel es muy útil para transformar espacios no linealmente separables en espacios linealmente separables, tan solo modificando la función kernel.



Con este cambio, ahora tenemos la capacidad de **clasificar datos no linealmente separables**.

Función de predicción

Recordemos que la función de predicción es: $\hat{y}_j = \text{sign}(\sum_{s=1}^S \alpha_s y_s (\mathbf{x}_s \cdot \mathbf{x}_j) + b)$.

Sin embargo, dado que el espacio está cambiando debido al kernel, la **función de predicción** también cambia.

$$\hat{y}_j = \text{sign} \left(\sum_{s=1}^S \alpha_s y_s K(\mathbf{x}_s, \mathbf{x}_j) + b \right)$$

- $\mathbf{x}_j$ es un nuevo punto.
- $\mathbf{x}_s$ son los vectores de soporte.
- S el número de observaciones de vectores de soporte.

Cálculo de b

Recordemos que $b = \frac{1}{S} \sum_{s=1}^S (y_s - \mathbf{w} \cdot \mathbf{x}_s)$.

Debido a que estamos cambiando el espacio, necesitamos incluir el kernel en el cálculo de b .

Reemplazando $\mathbf{w} = \sum_{s=1}^S \alpha_s y_s \mathbf{x}_s$ en b :

$$b = \frac{1}{S} \sum_{s=1}^S \left(y_s - \sum_{t=1}^S \alpha_t y_t (\mathbf{x}_t \cdot \mathbf{x}_s) \right)$$

Como antes, reescribimos $(\mathbf{x}_t \cdot \mathbf{x}_s)$ en función del kernel:

$$b = \frac{1}{S} \sum_{s=1}^S \left(y_s - \sum_{t=1}^S \alpha_t y_t K(\mathbf{x}_t, \mathbf{x}_s) \right)$$

- S es el número de los vectores de soporte.

Tipos de kernel

Existen diferentes **tipos de kernel** (Hastie et. al, 2008, p. 424):

- Kernel Lineal: $k(\mathbf{x}_i, \mathbf{x}_m) = \mathbf{x}_i \cdot \mathbf{x}_m$
- Kernel Polinómico de grado d : $k(\mathbf{x}_i, \mathbf{x}_m) = (1 + (\mathbf{x}_i \cdot \mathbf{x}_m))^d$

- Kernel de Base Radial o RBF: $k(\mathbf{x}_i, \mathbf{x}_m) = \exp(-\gamma \|\mathbf{x}_i - \mathbf{x}_m\|^2)$
- Kernel de Red Neuronal: $k(\mathbf{x}_i, \mathbf{x}_m) = \tanh(\lambda_1(\mathbf{x}_i \cdot \mathbf{x}_m) + \lambda_2)$

El libro de Kowalczy (2017, p. 76) el kernel polinómico tiene dos parámetros y se define así:
 $K(\mathbf{x} \cdot \mathbf{x}') = (\mathbf{x} \cdot \mathbf{x}' + c)^d$, donde c es un término constante y d el grado del kernel.

En el caso del **kernel polinómico**, si se utiliza un grado alto, se corre el riesgo de un sobreajuste. Si se elige un grado de 1 es simplemente el kernel lineal (Kowalczyk, p. 77)

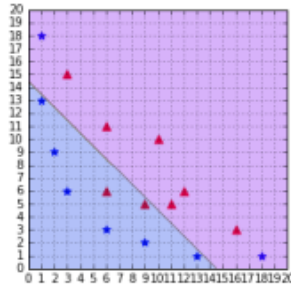


Figure 43: A polynomial kernel with degree = 1

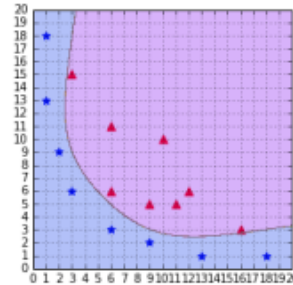


Figure 44: A polynomial kernel with degree = 6

En el caso del **kernel RBF**, si se elige un γ pequeño, el modelo se comporta como un kernel lineal. Si se elige un γ muy grande, el modelo es altamente influenciado por los vectores de soporte (Kowalczy, 2017, p. 80)

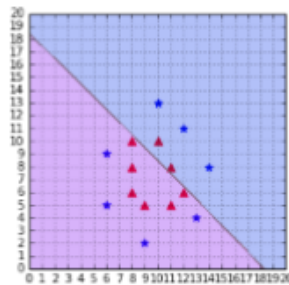


Figure 48: A Gaussian kernel with gamma = 1e-5

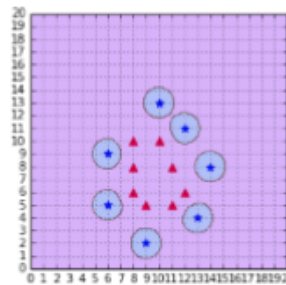


Figure 49: A Gaussian kernel with gamma = 2

El [artículo de Cesar Souza](#) describe 25 kernel.

Implementación SVM

En esta sección implementaremos la solución al problema dual del planteamiento *Soft margin* con el truco del kernel.

Programación Cuadrática o QP

Es un problema de optimización con una función objetivo cuadrática y restricciones de igualdad y desigualdad afines, según [cvxpy](#) la forma estándar común es:

$$\begin{aligned} \min_x \quad & \frac{1}{2}x^T Px + q^T x \\ \text{sujeto a} \quad & Gx \leq h \\ & Ax = b \end{aligned}$$

- $P \in \mathcal{S}_+^n$ es una matriz simétrica semidefinida positiva de tamaño n .
- $q \in \mathbb{R}^n$ es un vector de tamaño n .
- $G \in \mathbb{R}^{m \times n}$, $h \in \mathbb{R}^m$ son restricciones de desigualdad.
- $A \in \mathbb{R}^{p \times n}$, $b \in \mathbb{R}^p$ son restricciones de igualdad.
- $x \in \mathbb{R}^n$ es la variable de optimización.

El problema de optimización

Recordemos que el problema de optimización está en términos de maximización:

$$\begin{aligned} \max_{\alpha} \quad & \sum_{i=1}^n \alpha_i - \frac{1}{2} \sum_{i=1}^n \sum_{j=1}^n \alpha_i \alpha_j y_i y_j K(\mathbf{x}_i, \mathbf{x}_j) \\ \text{sujeto a} \quad & 0 \leq \alpha_i \leq C, \quad i = 1, \dots, n \\ & \sum_{i=1}^n \alpha_i y_i = 0 \end{aligned}$$

Sin embargo, para resolver este problema de optimización mediante QP, se requiere que la función objetivo esté en **términos de minimización**, para esto solo se multiplica por -1 :

$$\begin{aligned} \min_{\alpha} \quad & \frac{1}{2} \sum_{i=1}^n \sum_{j=1}^n \alpha_i \alpha_j y_i y_j K(\mathbf{x}_i, \mathbf{x}_j) - \sum_{i=1}^n \alpha_i \\ \text{sujeto a} \quad & 0 \leq \alpha_i \leq C, \quad i = 1, \dots, n \\ & \sum_{i=1}^n \alpha_i y_i = 0 \end{aligned}$$

expresado en **forma matricial**, tenemos:

$$\begin{aligned} \min_{\alpha} \quad & \frac{1}{2} \alpha^T ((\mathbf{y} \otimes \mathbf{y}) \odot \mathbf{K}) \alpha - \alpha \\ \text{sujeto a} \quad & -\alpha \leq 0, \quad \alpha \leq C \\ & \mathbf{y}^T \alpha = 0 \end{aligned}$$

- $\alpha = (\alpha_1, \dots, \alpha_n)$ es el vector de multiplicadores de Lagrange.
- $\mathbf{y} = (y_1, \dots, y_n)$ es el vector de respuestas.
- $\mathbf{K}$ es una matriz que contiene todos los posibles productos escalares de $\mathbf{x}_i$ (denominada matriz Gram ³), esto es así para kernel lineal.

Sin embargo, gracias al truco del kernel podemos modificar por otro tipo de operación.

- El operador $\otimes$ indica que es una operación *outer product* ⁴.
- El operador $\odot$ indica que es una multiplicación elemento a elemento.

Como se puede observar, este problema de optimización tiene una forma similar al problema de programación cuadrática.

Implementación con CVXOPT

La librería de python [cvxopt](#) requiere que el problema de optimización tenga la forma estándar de QP.

Resolviendo el problema dual

Preparación de los parámetros (matrices) $\mathbf{P}$, $\mathbf{q}$, $\mathbf{G}$, $\mathbf{h}$, $\mathbf{A}$, $\mathbf{b}$:

- función objetivo

$$\begin{aligned} \mathbf{P} &= (\mathbf{y}\mathbf{y}^T \mathbf{K})_{n \times n} \\ \mathbf{q} &= -\mathbf{1}_n \quad (\text{vector de unos}) \end{aligned}$$

- restricción de desigualdad

$$\mathbf{G} = \text{vstack}(-I_n, I_n) = \begin{pmatrix} -1 & 0 & \cdots & 0 \\ 0 & -1 & & \vdots \\ \vdots & & \ddots & 0 \\ 0 & \cdots & 0 & -1 \\ 1 & 0 & \cdots & 0 \\ 0 & 1 & & \vdots \\ \vdots & & \ddots & 0 \\ 0 & \cdots & 0 & 1 \end{pmatrix}_{2n \times n}$$

$$h = \text{hstack}(\mathbf{0}_n, \mathbf{C}_n) = (0 \quad \cdots \quad 0 \quad C \quad \cdots \quad C)_{2n}$$

- restricción de igualdad

$$A = \mathbf{y}_{1 \times n}$$

$$b = 0_1$$

A continuación se muestra el código:

```
from cvxopt import matrix, solvers
def solve_min_dual(X, y, C=0.1, kernel="linear", gamma=0.5, d=2):
    n = y.shape[0]
    # kernel
    K = calc_kernel(X, X, kernel, gamma, d)
    # Preparación de parámetros
    P = matrix(np.outer(y, y) * K)
    q = matrix(-np.ones(n))
    G = matrix(np.vstack([-np.eye(n), np.eye(n)]))
    h = matrix(np.hstack([np.zeros(n), C*np.ones(n)]))
    A = matrix(y, (1, n))
    b = matrix(np.zeros(1))

    solvers.options['show_progress'] = False
    solution = solvers.qp(P, q, G, h, A, b)

    # multiplicadores de lagrange (alphas)
    return np.ravel(solution['x'])
```

Kernels

```
def calc_kernel(X1, X2, type="linear", gamma=0.5, d=2):  
  
    K = np.zeros((X1.shape[0], X2.shape[0]))  
  
    if type == "linear":  
        for i, xi in enumerate(X1):  
            for j, xm in enumerate(X2):  
                K[i, j] = np.dot(xi, xm)  
  
    elif type == "radial":  
        for i, xi in enumerate(X1):  
            for j, xm in enumerate(X2):  
                K[i, j] = np.exp(-gamma*np.linalg.norm(xi-xm)**2)  
  
    elif type == "poly":  
        for i, xi in enumerate(X1):  
            for j, xm in enumerate(X2):  
                K[i, j] = (1 + np.dot(xi, xm)) ** d  
  
    return K
```

Parámetros w y b

Cálculo de w :

$$\mathbf{w} = \sum_{i=1}^n \alpha_i y_i \mathbf{x}_i$$

```
def compute_w(alphas, X, y):  
    return X.T @ (alphas * y)
```

Cálculo de b :

$$b = \frac{1}{S} \sum_{s=1}^S \left(y_s - \sum_{t=1}^S \alpha_t y_t K(\mathbf{x}_t, \mathbf{x}_s) \right)$$

```
def compute_b(X_sv, y_sv, alphas_sv, kernel="linear", gamma=0.5, d=2):  
    b = (y_sv) - np.sum(alphas_sv * y_sv *  
                        calc_kernel(X_sv, X_sv, kernel, d=d), axis=1)  
    return np.mean(b)
```

Vectores de soporte

```
# multiplicadores de Lagrange
alphas = solve_min_dual(X, y, C=0.5, kernel="linear")

# vectores de soporte (alpha > 0)
support_vectors = alphas > 1e-5
alphas_sv = alphas[support_vectors]
X_sv = X[support_vectors]
y_sv = y[support_vectors]
```

Función de predicción

$$\hat{y}_j = \text{sign} \left(\sum_{s=1}^S \alpha_s y_s K(\mathbf{x}_s, \mathbf{x}_j) + b \right)$$

- $\mathbf{x}_j$ es la nueva observación.
- $\mathbf{x}_s$ son los vector de soporte.

```
def predict(X_new, X_sv, y_sv, alphas_sv, b,
            kernel="linear", gamma=0.5, d=2):
    K = calc_kernel(X_new, X_sv, kernel, gamma, d)
    f = np.sum(alphas_sv * y_sv * K, axis=1) + b
    return np.sign(f)
```

Referencias

- Hastie, Tibshirani, Friedman, 2008, The Elements of Statistical Machine Learning.
- Kowalczyk, Alexandre. Support Vector Machines Succinctly, 2017
- Support Vector Machines for Classification - Oscar Contreras (2019) <https://medium.com/data-science/support-vector-machines-for-classification-fc7c1565e3>
- CVXOPT Documentation <https://cvxopt.org/userguide/coneprog.html#quadratic-programming>

Anexos

Principio de dualidad, tells us that an optimization problem can be viewed from two perspectives. The first one is the primal problem, and the other one is the dual problem, En el caso de SVM, resolver el dual es lo mismo que resolver el primal, ya que cumple dos condiciones: es un *problema de optimización convexa* y cumple con la condición de Slater. (Kowalczy, 2017, p. 55-56)

-
1. Un **hiperplano óptimo** es aquel hiperplano que mejor separa los datos (p. 39) [↵](#)
 2. El objetivo de un **problema de optimización** es minimizar o maximizar una función con respecto a una variable. [↵](#) [↵](#)
 3. Una matriz Gram semidefinida positiva, esta matriz contiene todos los productos escalares de un conjunto de vectores. Se define de la siguiente manera: Dado un conjunto de vectores $\mathbf{v}_1, \dots, \mathbf{v}_n$, cada elemento de la matriz Gram $\mathbf{G}$ se calcula mediante $G_{ij} = \mathbf{v}_i \cdot \mathbf{v}_j$. [↵](#)
 4. El outer product o producto exterior es una operación en álgebra lineal que toma dos vectores y produce una matriz. Se denota como $\mathbf{u} \otimes \mathbf{v} = \mathbf{u}\mathbf{v}^T$, donde cada elemento de la matriz resultante se calcula como $W_{ij} = u_i * v_j$. [↵](#)

By Carlos Ramos (2026)